

Lab 1 – Lab su spoofing TCP/IP

Sicurezza dei Sistemi e delle reti

Modulo 4 - Protocolli TCP

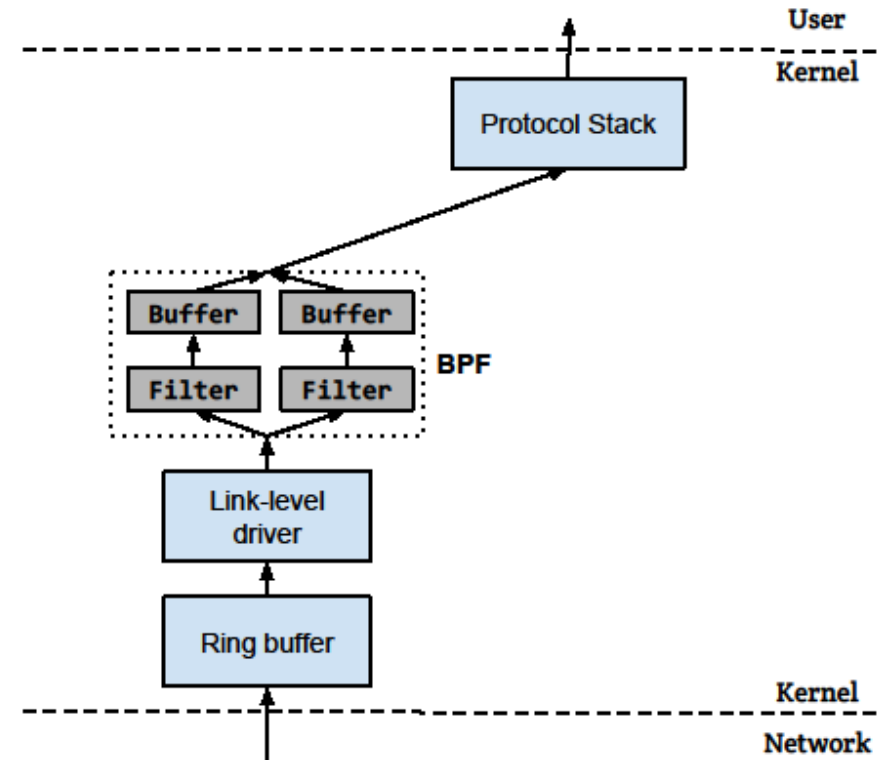
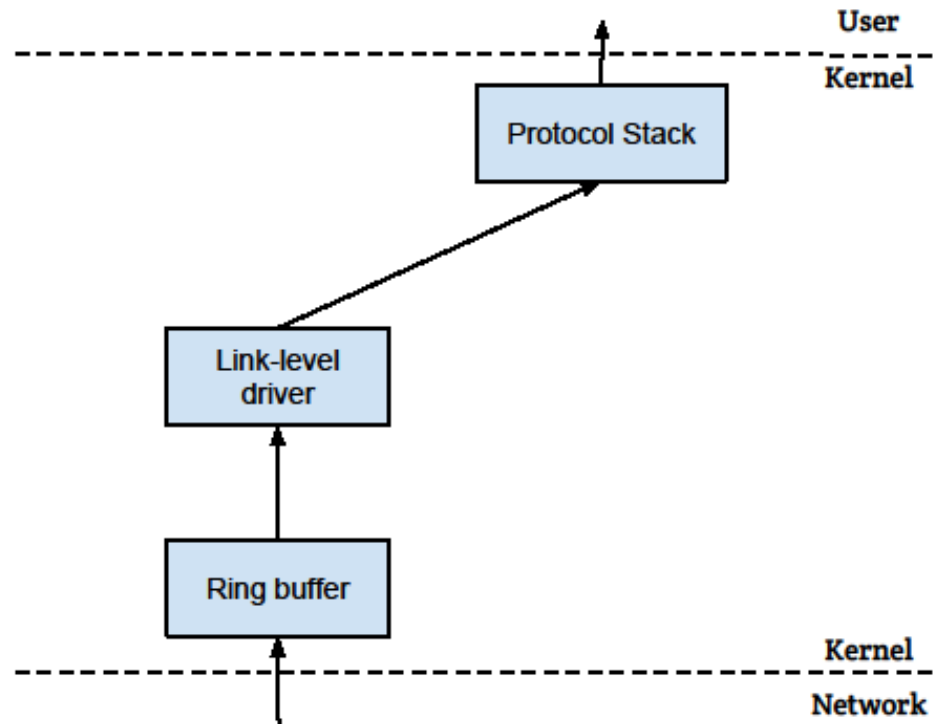
Unità didattica 2 – Vulnerabilità di TCP/IP

Stelvio Cimato

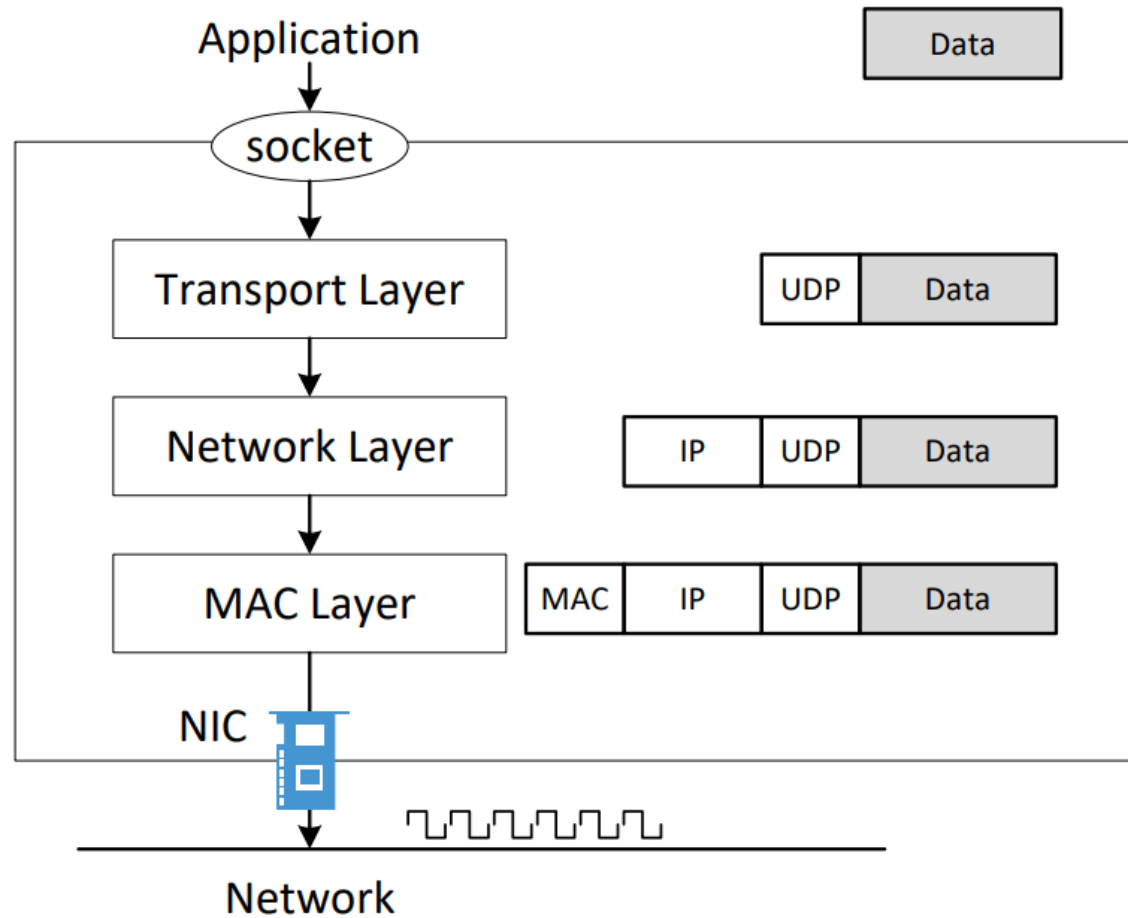
How Packets Are Received

- **NIC (Network Interface Card)** è un collegamento fisico o logico tra una macchina e una rete
- **Ogni NIC ha un indirizzo MAC**
- **Ogni NIC sulla rete riceverà tutti i frame sul cavo**
- **NIC controlla l'indirizzo di destinazione per ogni pacchetto,**
 - se l'indirizzo corrisponde all'indirizzo MAC della scheda, viene ulteriormente copiato in un buffer nel kernel
- **I frame che non sono destinati a una determinata NIC vengono scartati**
- **Quando opera in modalità promiscua, il NIC passa ogni frame ricevuto dalla rete al kernel**
- **Se un programma sniffer è registrato con il kernel, sarà in grado di vedere tutti i pacchetti**
- **In Wi-Fi, si chiama Modalità monitor**

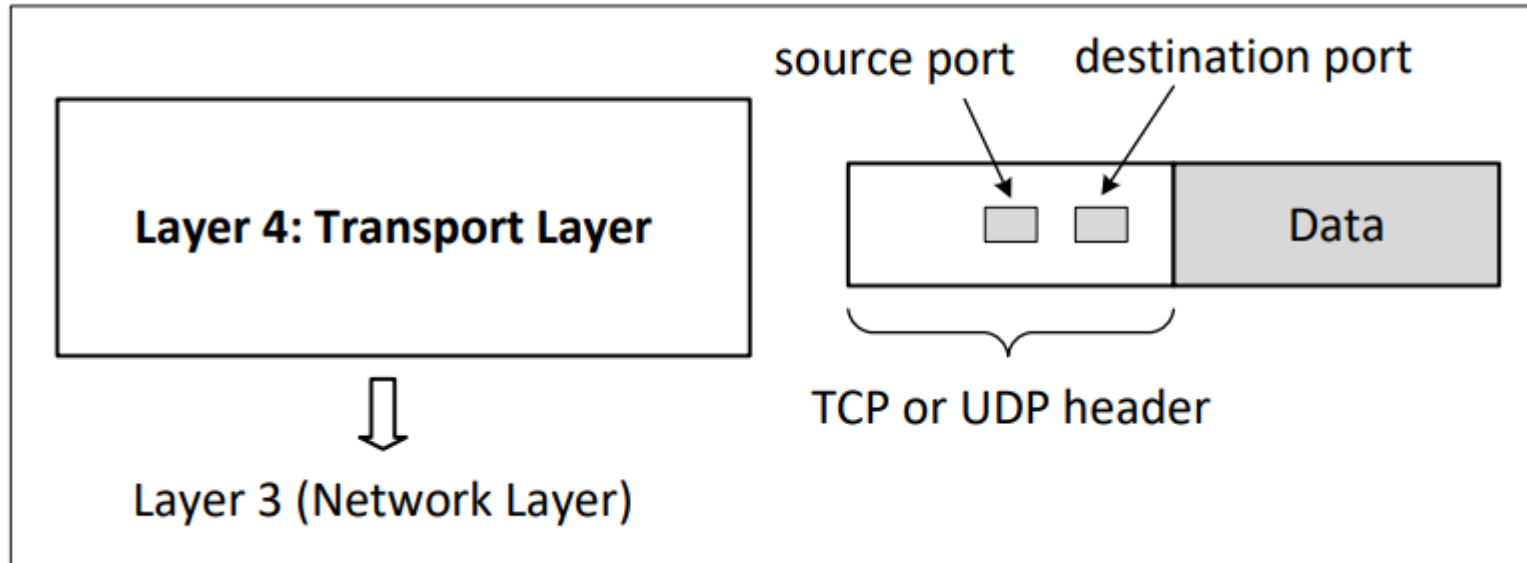
How packets are received



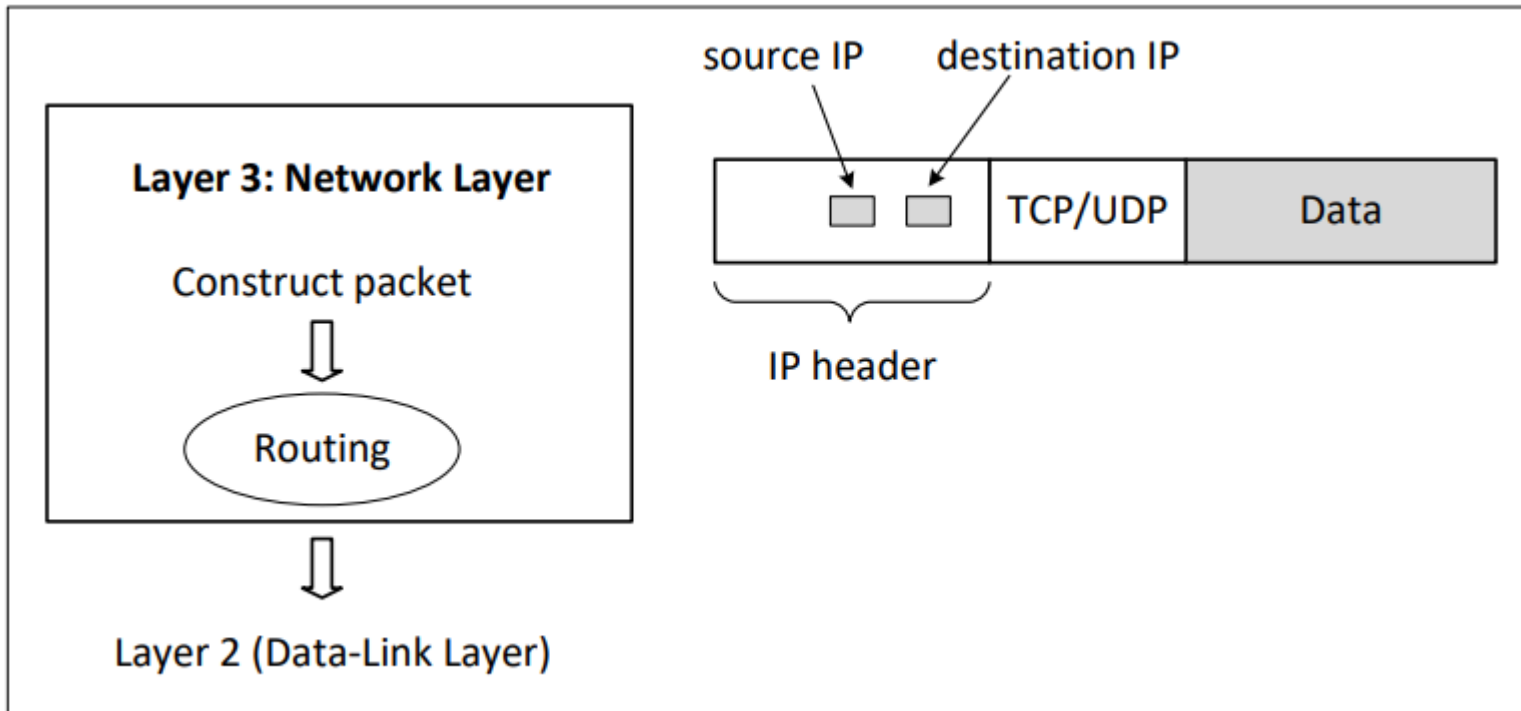
How Packets Are Constructed



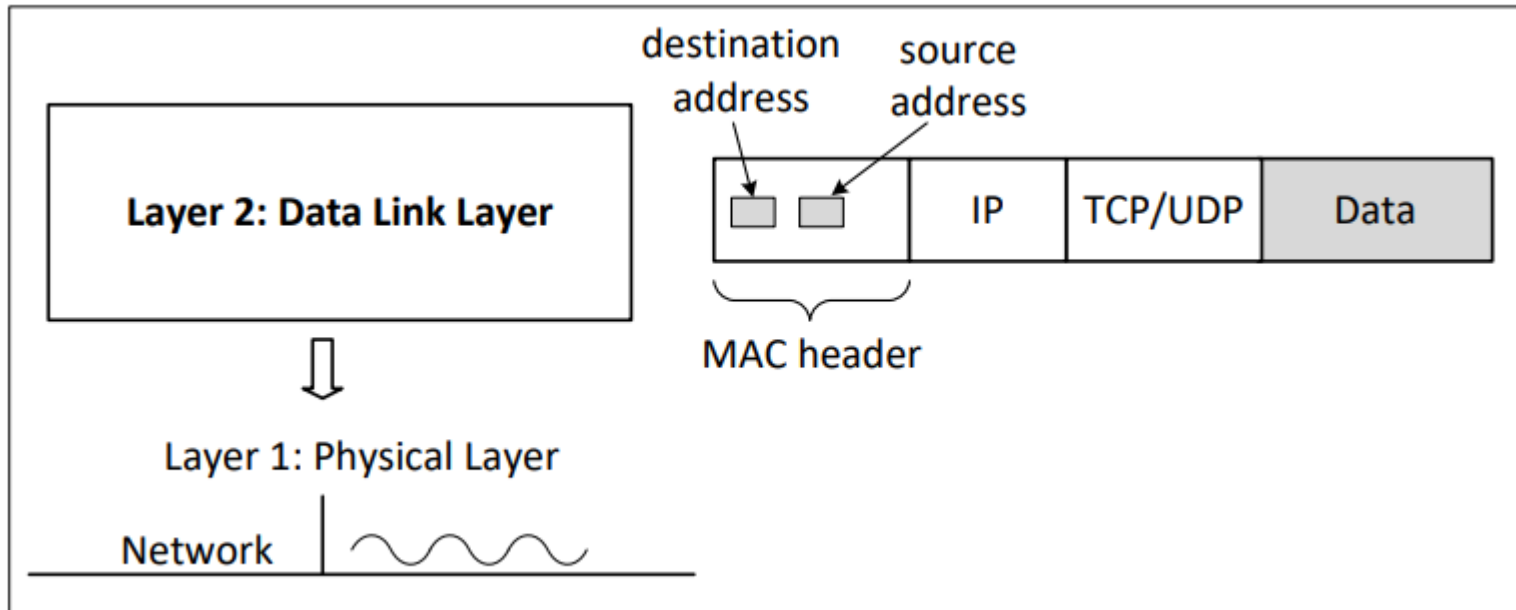
Layer 4: Transport Layer



Layer 3: Network Layer



Layer 2: Data Link Layer (MAC Layer)



Packet Sending Tools

- **Using netcat**

```
$ nc <ip> <port>      ← send out TCP packet  
$ nc -u <ip> <port>   ← send out UDP packet
```

```
$ echo "data" > /dev/udp/<ip>/<port>  
$ echo "data" > /dev/tcp/<ip>/<port>
```


What is Scapy

- **Scapy è un potente programma interattivo di manipolazione dei pacchetti.**
- **Scritto in Python**
- **Utile per**
 - falsificare o decodificare i pacchetti
 - mandarli sulla rete,
 - catturarli
- **Può gestire facilmente le attività più classiche come la scansione, il tracerouting, gli attacchi o il rilevamento della rete**
- **<https://scapy.net/>**

Sending Packet in Python (1)

- **UDP Client**

```
#!/usr/bin/python3
```

```
import socket
```

```
IP    = "127.0.0.1"
```

```
PORT = 9090
```

```
data = b'Hello, World!'
```

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
sock.sendto(data, (IP, PORT))
```

Sending Packet in Python (1)

- **Execution Results**

```
$ nc -l uv 9090  
Listening on [0.0.0.0] (family 0, port 9090)  
Hello, World!█
```

Receiving Packets in Python

- **UDP Server**

```
#!/usr/bin/python3

import socket

IP    = "0.0.0.0"
PORT = 9090

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((IP, PORT))

while True:
    data, (ip, port) = sock.recvfrom(1024)
    print("Sender: {} and Port: {}".format(ip, port))
    print("Received message: {}".format(data))
```

UDP Server

```
Terminal
seed@10.0.2.6:$ nc -u 10.0.2.7 9090
hello
hello again
█
```

```
Terminal
Server(10.0.2.7):$ udp_server.py
Sender: 10.0.2.6 and Port: 49112
Received message: b'hello\n'
Sender: 10.0.2.6 and Port: 49112
Received message: b'hello again\n'
█
```

Using Tools to Sniff and Spoof Packets

- **Per usare Scapy, possiamo scrivere un programma Python e quindi eseguire questo programma usando Python.**
 - **Dovremmo eseguire Python usando il privilegio di root perché il privilegio è necessario per lo spoofing del pacchetto**
 - **Puoi eseguire Python o utilizzare la modalità interattiva di Python**
- `#!/bin/bin/python`
 - `Print («SNIFFO»)`
 - `from scapy.all import *`
 - `a = IP()`
 - `a.show()`
-
- `interactive mode of Python`

Sniffing

- **def print_pkt(pkt):**
 - **pkt.show()**
 - **pkt = sniff(filter='icmp',prn=print_pkt)**
- **Il filtro di Scapy utilizza la sintassi BPF (Berkeley Packet Filter);**
 - **Cattura solo il pacchetto ICMP**
 - **Cattura qualsiasi pacchetto TCP proveniente da un particolare IP e con un numero di porta di destinazione 23.**
 - **Cattura i pacchetti provenienti da o destinati a una particolare sottorete.**
 - **Spoofing ICMP Packets**

Scapy Example 1

```
#!/usr/bin/python3

from scapy.all import *

pkt = sniff(iface='enp0s3',
            filter='icmp or udp',
            count=10)

pkt.summary()
```

```
seed@VM:~$ ip -br addr
lo                UNKNOWN      127.0.0.1/8 ::1/
enp0s3            UP           10.0.5.5/24 fe80:
docker0           DOWN        172.17.0.1/16 fe
br-54cdc1101b1a   UP           10.9.0.1/24 fe80:
br-0569b491802f   UP           192.168.60.1/24
```

```
root@9eb2c057887f:~# ip -br addr
lo                UNKNOWN      127.0.0.1/8
eth0@if1882       UP           10.9.0.5/24
```


Scapy Example 2

```
#!/usr/bin/python3

from scapy.all import *

def process_packet(pkt):
    #hexdump(pkt)
    pkt.show()
    print("-----")

f = 'udp and dst portrange 50-55 or icmp'

sniff(iface='enp0s3', filter = f, prn=process_packet)
```

Filter Examples for Scapy

- **Berkeley Packet Filter (BPF) syntax**
- **Same as tcpdump**

```
dst host 10.0.2.5: only capture the packets going to 10.0.2.5.  
src host 10.0.2.6: only capture the packets coming from 10.0.2.6.  
host 10.0.2.6 and src port 9090: only capture the packets coming  
from or going to 10.0.2.6 with the source port being 9090.  
tcp: only capture TCP packets.
```

Scapy: Display Packets

- Using `hexdump()`

```
>>> hexdump(pkt)
0000  52 54 00 12 35 00 08
0010  00 54 F2 29 40 00 40
0020  08 08 08 00 98 01 10
0030  0C 00 08 09 0A 0B 0C
0040  16 17 18 19 1A 1B 1C
0050  26 27 28 29 2A 2B 2C
0060  36 37
```

- Using

```
>>> pkt.show()
###[ Ethernet ]###
    dst      = 52:54:00:12:35:00
    src      = 08:00:27:77:2e:c3
    type     = IPv4
###[ IP ]###
    version  = 4
    ihl      = 5
    ...
    proto    = icmp
    checksum = 0x3c9a
    src      = 10.0.2.8
    dst      = 8.8.8.8
    \options \
###[ ICMP ]###
```

A Sniffer Example

```
def process_packet(pkt):
    if pkt.haslayer(IP):
        ip = pkt[IP]
        print("IP: {} --> {}".format(ip.src, ip.dst))

    if pkt.haslayer(TCP):
        tcp = pkt[TCP]
        print("    TCP port: {} --> {}".format(tcp.sport, tcp.dport))

    elif pkt.haslayer(UDP):
        udp = pkt[UDP]
        print("    UDP port: {} --> {}".format(udp.sport, udp.dport))

    elif pkt.haslayer(ICMP):
        icmp = pkt[ICMP]
        print("    ICMP type: {}".format(icmp.type))

    else:
        print("    Other protocol")

sniff iface='enp0s3', filter='ip', prn=process_packet)
```

Spoofing ICMP Packets

- Spoofing di pacchetti IP con un indirizzo IP di origine arbitrario.
- Spoofing ICMP echo request packets e inviarli a un'altra VM sulla stessa rete
- `a = IP()`
- `>>> a.dst = '10.0.2.3'`
- `>>> b = ICMP()`
- `>>> p = a/b`
- `>>> send(p)`
- crea un oggetto IP dalla classe IP;
- un attributo di classe è definito per ogni campo di intestazione IP.
- `ls(a)` o `ls(IP)` per vedere tutti i nomi/valori degli attributi.
- Possiamo anche usare `a.show()` e `IP.show()` insieme per formare un nuovo oggetto.
- L'operatore `/` fa overload dalla classe IP, significa aggiungere come campo payload di a e modificare i campi di conseguenza.

```
#!/usr/bin/python3
from scapy.all import *

print("SENDING SPOOFED ICMP PACKET.....")
ip = IP(src="1.2.3.4", dst="93.184.216.34")
icmp = ICMP()
pkt = ip/icmp
pkt.show()
send(pkt, verbose=0)
```

Sniffing and-then Spoofing

- **Due macchine virtuali sulla stessa LAN (o in ambiente docker).**
- **Da VM A, ping IP X.**
- **Se X è attivo, il programma Ping riceverà una risposta eco e stamperà la risposta.**
- **Il tuo programma sniff-and-then-spoof viene eseguito su VM B, che monitora la LAN tramite lo sniffing dei pacchetti.**
- **Ogni volta che vede una richiesta echo ICMP, indipendentemente da quale sia l'indirizzo IP di destinazione, il tuo programma dovrebbe inviare immediatamente una risposta echo utilizzando la tecnica di spoofing dei pacchetti.**
- **Pertanto, indipendentemente dal fatto che la macchina X sia attiva o meno, il programma ping riceverà sempre una risposta che indica che X è attiva.**

Sniff Request and Spoof Reply: Code

```
def spoof_pkt(pkt):
    if ICMP in pkt and pkt[ICMP].type == 8:
        print("Original Packet.....")
        print("Source IP : ", pkt[IP].src)
        print("Destination IP :", pkt[IP].dst)

        ip = IP(src=pkt[IP].dst, dst=pkt[IP].src,
                ihl=pkt[IP].ihl, ttl = 99)
        icmp = ICMP(type=0, id=pkt[ICMP].id, seq=pkt[ICMP].seq)
        data = pkt[Raw].load
        newpkt = ip/icmp/data

        print("Spoofed Packet.....")
        print("Source IP : ", newpkt[IP].src)
        print("Destination IP :", newpkt[IP].dst)

        send(newpkt, verbose=0)

pkt = sniff(iface = 'br-54cdc1101b1a',
            filter = 'icmp and src host 10.9.0.5',
            prn = spoof_pkt)
```

Example: implement ping

```
#!/usr/bin/python3
from scapy.all import *

ip = IP(dst="8.8.8.8")
icmp = ICMP()
pkt = ip/icmp
reply = sr1(pkt)
print("ICMP reply .....")
print("Source IP : ", reply[IP].src)
print("Destination IP :", reply[IP].dst)
```


Traceroute Code

```
b = ICMP()  
a = IP()  
a.dst = '93.184.216.34'  
  
TTL = 3  
a.ttl = TTL  
h = sr1(a/b, timeout=2, verbose=0)  
if h is None:  
    print("Router: *** (hops = {})".format(TTL))  
else:  
    print("Router: {} (hops = {})".format(h.src, TTL))
```